HW 2B

A secure login for a Juice Shop-style shop

Secure feature design, a login form with checks on both sides, and an attempt to break that form.

Part 1. Secure feature design

Juice Shop is published with three flaws a registration system has to close.

The login query is open to SQL injection. A crafted email can change the statement and skip the password check, which is an authentication bypass. Binding that email to a placeholder (WHERE email = ?) keeps it in a value slot, so the database does not parse it as SQL. Rejecting values that are not email addresses stops the obvious strings before they are bound.

Search results and product reviews are open to cross-site scripting. A script planted there runs in the next shopper's browser and can act with their session. Writing untrusted data as text, and sending a Content-Security-Policy that disallows inline scripts, keeps the browser from running it.

Registration and password reset fail open when the secret is weak or the check lives only in the page. The server has to decide identity. Passwords should be stored as slow hashes. bcrypt.hashSync(password, 10) writes a salted hash, and bcrypt.compareSync(password, hash) checks a guess without reversing it. A stolen table does not reveal the password, and the cost slows guessing.

```
const findUser = db.prepare(
  "SELECT email, password_hash FROM users WHERE email = ?"
);
const user = findUser.get(normalized);
const hash = user ? user.password_hash : dummyHash;
const match = bcrypt.compareSync(password, hash);

// registration stores the hash, never the password
insertUser.run(normalized, bcrypt.hashSync(password, 10));
```

The lab form in Parts 2 and 3 uses these three controls: a bound email parameter, text output, and a bcrypt hash.

Part 2. Login form

I built a Juice Shop-style login in HTML, CSS, and JavaScript, plus a registration page. validateLogin, in public/validate.js, rejects an empty submit, requires "@" in the email, and requires at least 8 characters in the password. The browser runs that function before fetch. server.js imports the same function for POST /api/login and POST /api/register, so skipping the page script does not skip the rules.

The server stores passwords with bcrypt at cost 10 and inserts only the hash. The account lookup uses a bound parameter. Errors, and the email that was typed, are placed with textContent. A successful login sets an HttpOnly, SameSite=Strict cookie. The page can show the email and cannot read the cookie.

From this project, run npm install, then npm start, and open http://127.0.0.1:47291. The lab account is

demo@juice.lab / Lemonade1. npm test covers the same checks. README.md is the runbook.

GitHub repository: publish this project as a public repository and use that URL on the submitted copy. This workspace has no public GitHub remote, so a live link is not printed here. The HTML, JavaScript, server, and README are the files to publish.

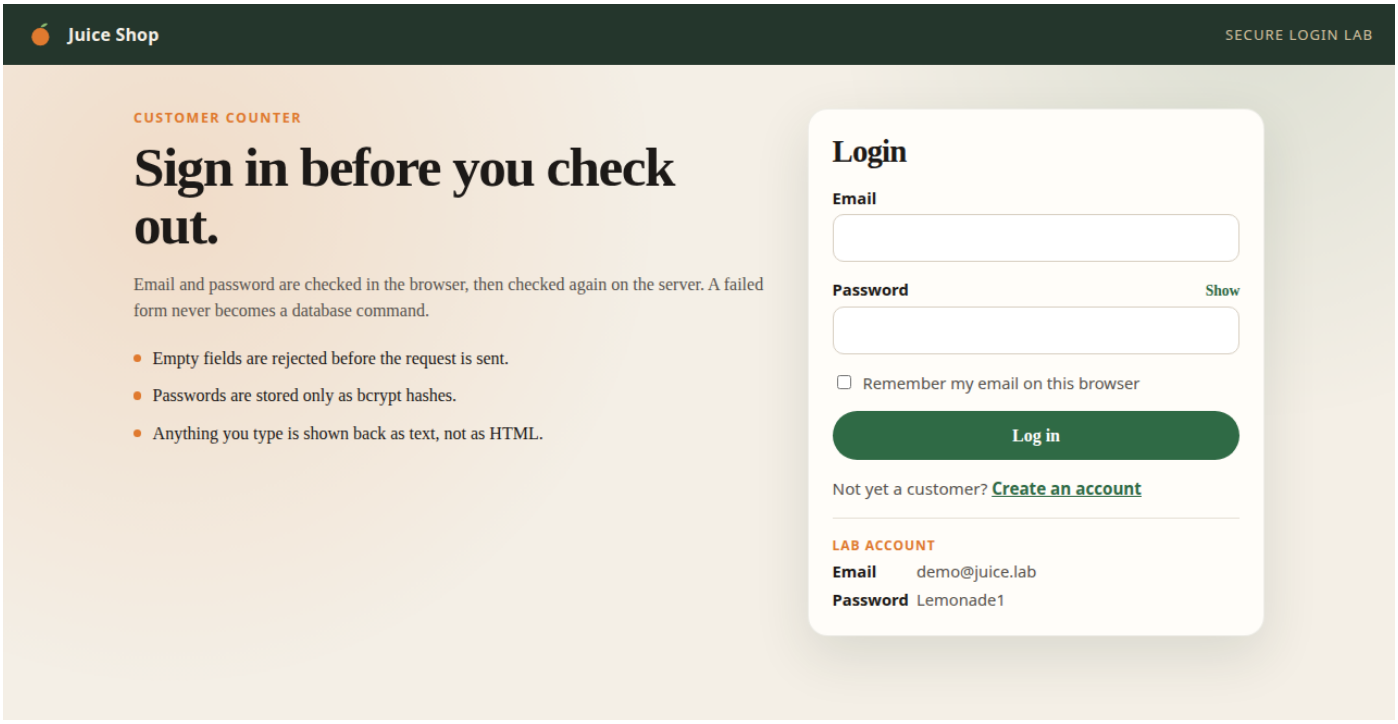


Figure 1. Login page. Email, password, and the lab account are on the card.

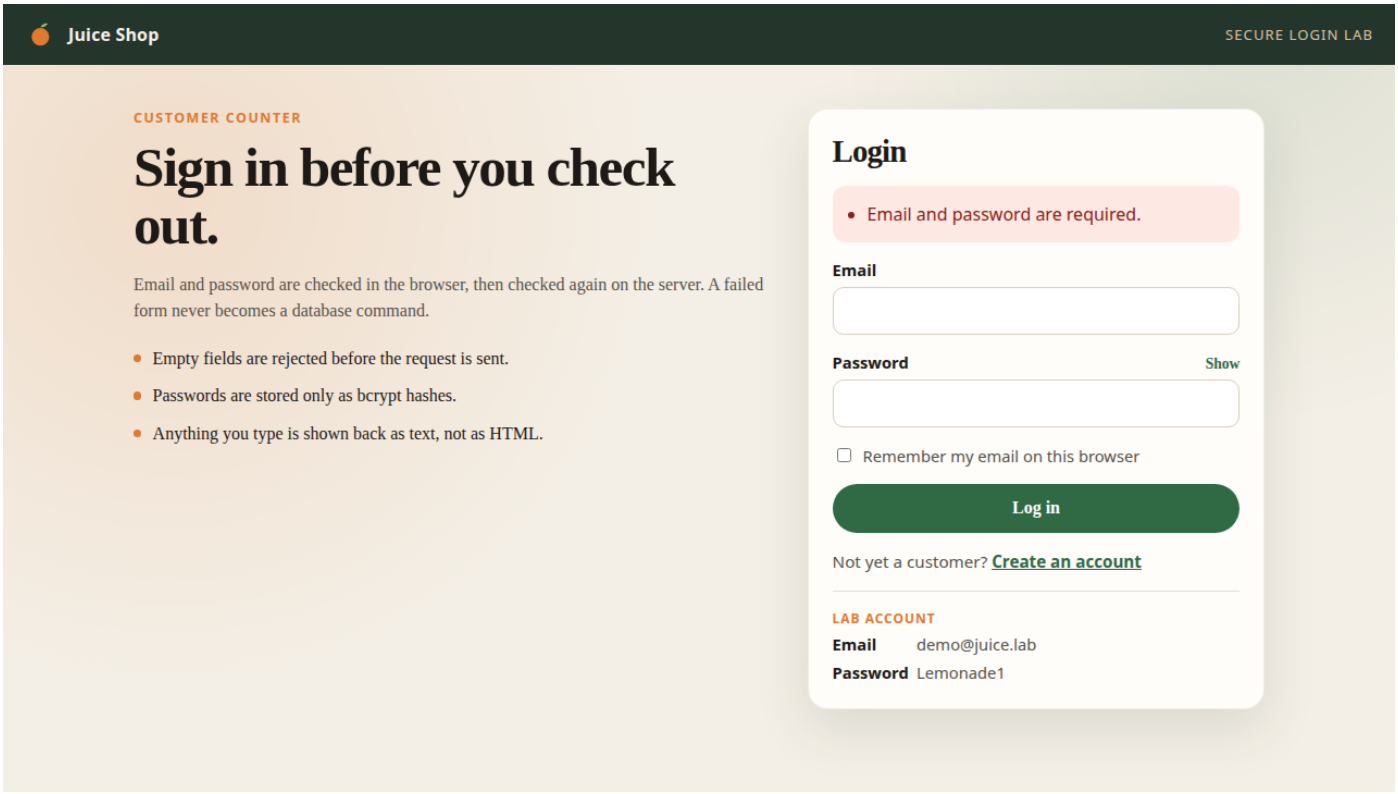


Figure 2. An empty submit is stopped in the browser: email and password are required.

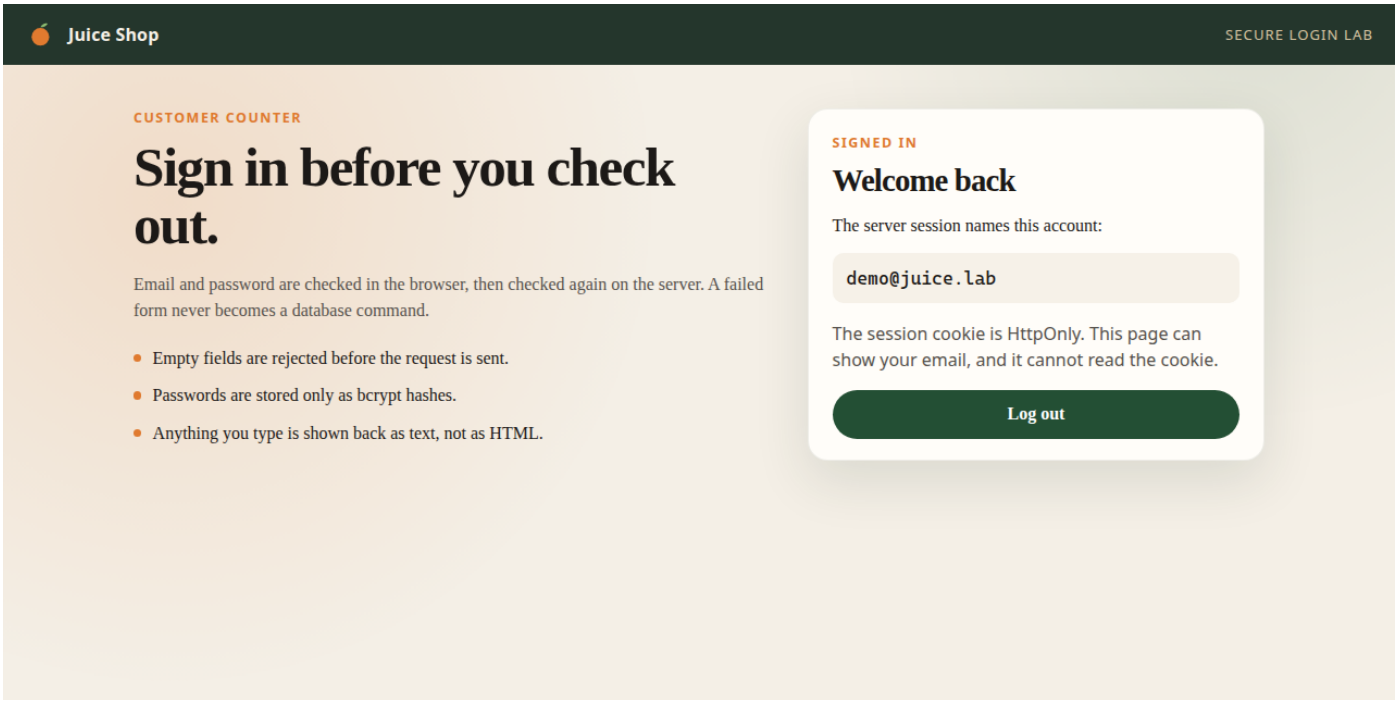


Figure 3. After a valid login the server session names the account. The cookie is HttpOnly.

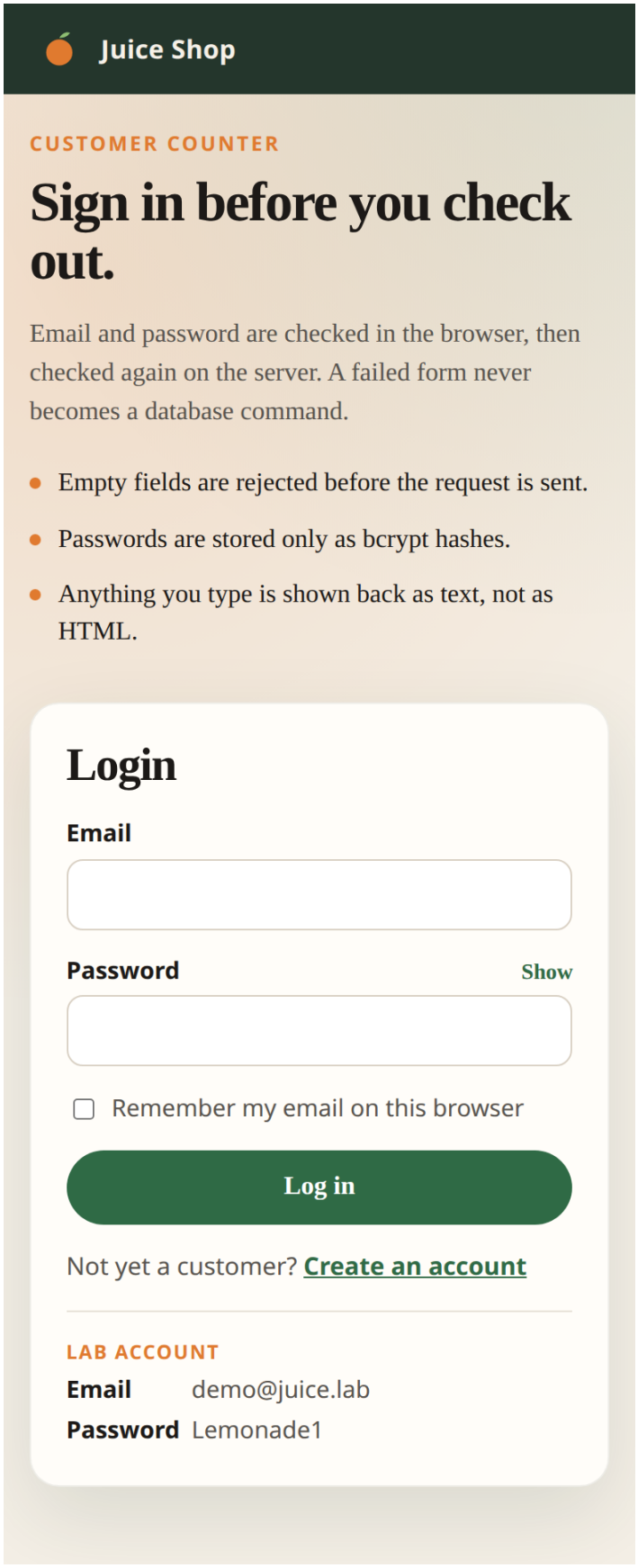


Figure 4. The same login stacked on a narrow screen.

Part 3. Trying to break the form

I tried to break the login I had just built, at http://127.0.0.1:47291.

1. I submitted the form with both fields empty. The page said "Email and password are required." The login request was not sent.

2. I entered demo@juice.lab and the password short. The page said the password must be at least 8 characters, and again did not call the server.

3. I skipped the page script and posted a classic SQL login string, a quote followed by OR 1=1 and a comment, as the email. The server returned 400 because the value has no "@". I then submitted an email-shaped string, demo@juice.lab'OR'1'='1, with the password password1. The server returned 401, "Invalid email or password." The lab account stayed locked. That string is bound to the ? placeholder and compared as an address. It is not concatenated into the SQL.

4. I typed "><svg/onload=alert(1)>@x.co as the email and Lemonade1 as the password. No alert ran. The same characters showed up in the field and in the block labeled "printed as text." The API response did not echo the markup. The page is served with script-src 'self'.

The attack did not succeed. The SQL attempt is stopped by the bound parameter, with the email check in front of it. The markup attempt is stopped by textContent. If that string had been assigned with innerHTML, the browser would have run the onload handler. Leaving it on textContent, and keeping the policy that blocks inline scripts, is the fix.

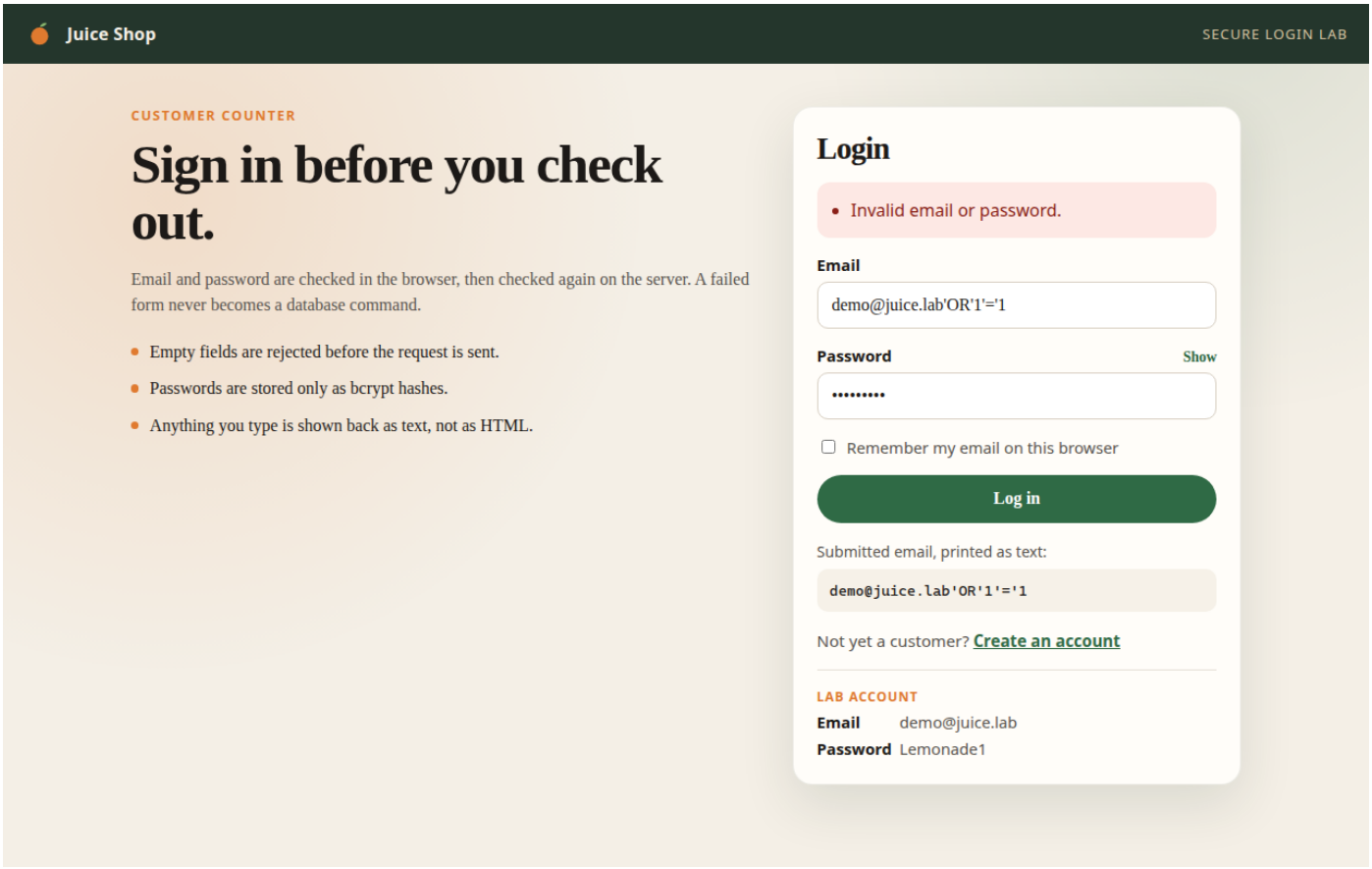


Figure 5. An email-shaped SQL fragment is printed as text and rejected. Sign-in does not succeed.

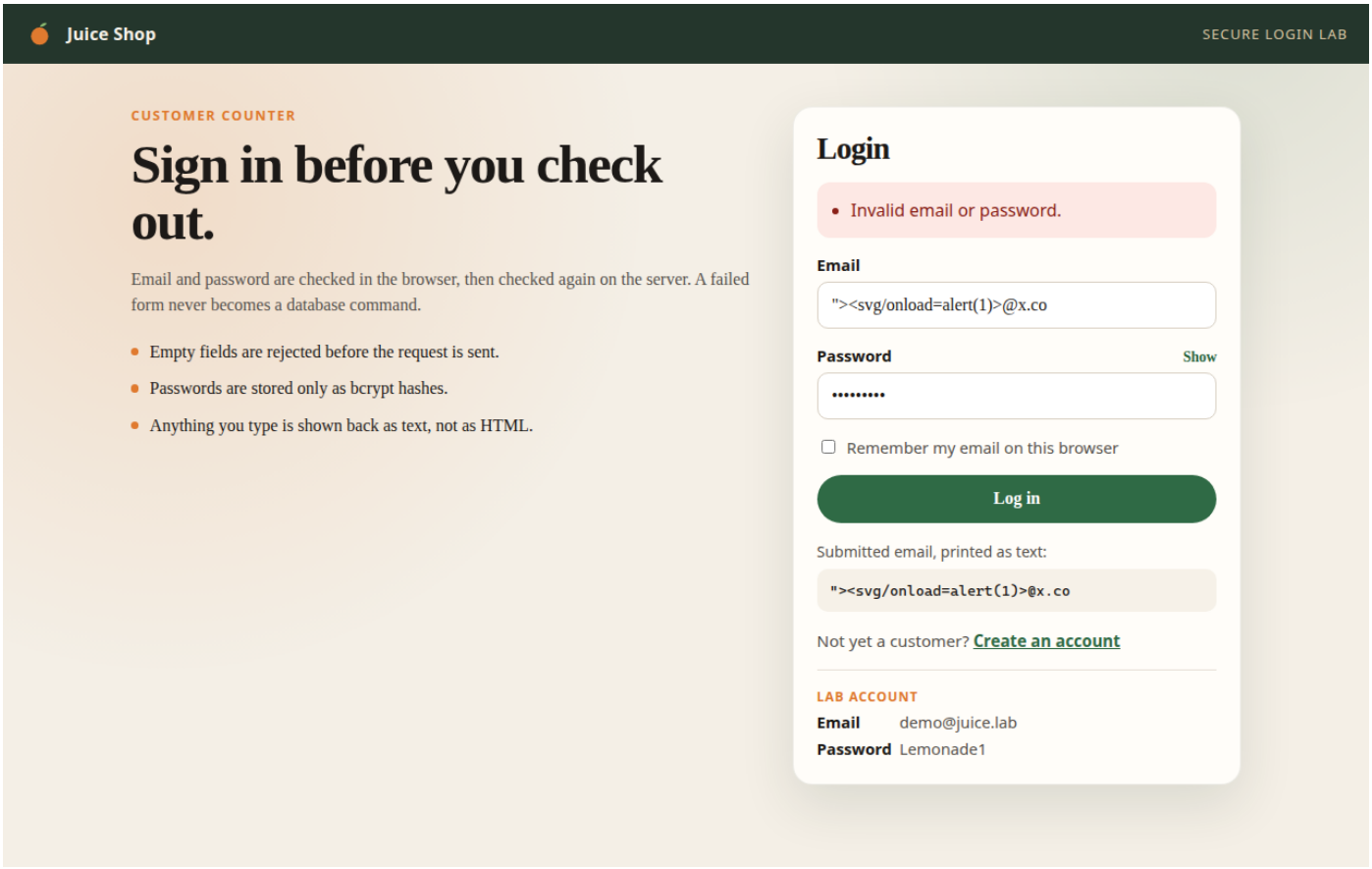


Figure 6. A markup-shaped email is shown as text. No script dialog ran.